

程序设计 II 第 5 周 题目汇总（完整版）

来源：Matrix · 课程「程序设计 II（25 级 曾坤 计算机学院）」含：完整题面、提交文件、平台提供的初始文件（.hpp/main.cpp 等）。选择题含标准答案。

共 7 项

25 程设 II-周 5-课后 1

类型：编程题

题面

学生信息类 Student

实现一个 Student 类，表示学生信息。

类的要求：

1. 包含三个私有成员：
 - const int id：学生学号（常量，一旦初始化不可修改）
 - const string name：学生姓名（常量，一旦初始化不可修改）
 - const int& score：引用，指向外部传入的成绩（必须用初始化器列表）
2. 提供构造函数 Student(int id, string name, int& score)：
 - 使用**成员初始化器列表**初始化 id、name 和 score
 - 输出信息："Student created: [姓名] ([学号]), score reference initialized"
3. 提供成员函数 void print() const：
 - 输出学生信息，格式："[姓名] ([学号]), score: [成绩]"
4. 提供成员函数 void printScore() const：
 - 输出当前成绩："Current score: [成绩]"

输入数据

第一行，一个整数 id，表示学号 第二行，一个字符串 name，表示姓名 第三行，一个整数 score，表示成绩

输出数据

第一行，构造函数输出的创建信息 第二行，学生信息 第三行，当前成绩 第四行，空行 第五行，=== After score update === 第六行，修改后的当前成绩

输入示例

10001
Alice
95

输出示例

Student created: Alice (10001), score reference initialized
Alice (10001), score: 95
Current score: 95

=== After score update ===
Current score: 100

提交文件：Student.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include <string>
#include "Student.h"
using namespace std;

int main() {
    int id;
    string name;
    int score;

    cin >> id >> name >> score;

    // 创建Student 对象
    Student stu(id, name, score);

    // 输出学生信息
    stu.print();

    // 输出当前成绩
    stu.printScore();

    // 修改成绩
    score = 100;
    cout << "\n=== After score update ===" << endl;
    stu.printScore();

    return 0;
}
```

文件 Student.h：

```

#ifndef STUDENT_H
#define STUDENT_H

#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    const int id;           // 常量成员，必须使用初始化器列表
    const string name;      // 常量成员，必须使用初始化器列表
    const int& score;        // 引用成员，必须使用初始化器列表

public:
    // 构造函数：使用成员初始化器列表
    Student(int id, string name, int& score);

    // 打印学生信息
    void print() const;

    // 打印当前成绩
    void printScore() const;
};

#endif

```

25 程设 II-周 5-课后 2

类型：编程题

题面

自定义字符串类（MyString）

实现一个 MyString 类，用于管理动态分配的字符数组。

类的要求：

- 包含两个私有成员：
 - char* data：指向动态分配的字符数组
 - int len：字符串的长度（不包括结尾的 \0）
- 提供构造函数 MyString(const char* str)：
 - 使用 new[] 为 data 分配足够的内存（长度为 strlen(str) + 1）
 - 将传入的字符串拷贝到动态分配的空间中

- 输出信息："MyString created: [字符串]"
- 3. 提供析构函数 ~MyString()：
 - 使用 delete[] 释放 data 指向的内存
 - 输出信息："MyString destroyed: [字符串]"
- 4. 提供成员函数 void print() const：
 - 输出字符串内容
- 5. 提供成员函数 int length() const：
 - 返回字符串的长度
- 6. 提供成员函数 char get(int index) const：
 - 如果 index 越界则输出 "Index out of range!"
 - 否则返回指定位置的字符（索引从 0 开始）

输入数据

第一行，一个整数 n ，表示测试用例的数量 ($1 \leq n \leq 10$)

接下来 n 组测试数据，每组包含：

- 第一行，一个字符串 str，表示要创建的字符串内容
- 第二行，一个整数 k ，表示接下来有 k 个查询
- 接下来 k 行，每行一个整数 idx，表示要查询的字符位置

输出数据

对于每个测试用例：

- 首先输出构造函数创建的字符串信息
- 然后输出字符串内容
- 输出字符串长度
- 对于每个查询，输出对应位置的字符（如果越界则输出错误信息）
- 最后输出析构函数释放的信息

输入示例

```
2
Hello
5
0
1
2
3
10
World
3
0
```

5
2

输出示例

MyString created: Hello
Hello
Length: 5
Character at index 0: H
Character at index 1: e
Character at index 2: l
Character at index 3: l
Index out of range!
MyString destroyed: Hello
MyString created: World
World
Length: 5
Character at index 0: W
Index out of range!
Character at index 2: r
MyString destroyed: World

提交文件：MyString.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include <string>
#include "MyString.h"
using namespace std;

int main() {
    int n;
    cin >> n;

    for (int t = 0; t < n; t++) {
        string str;
        cin >> str;

        MyString s(str.c_str());
        s.print();
        cout << "Length: " << s.length() << endl;

        int k;
        cin >> k;

        for (int i = 0; i < k; i++) {
            int idx;
            cin >> idx;
            char ch = s.get(idx);
```

```

        if (ch != '\0') {
            cout << "Character at index " << idx << ": " << ch << endl;
        }
    }
}

return 0;
}

```

文件 MyString.h :

```

#ifndef MYSTRING_H
#define MYSTRING_H

#include <iostream>
using namespace std;

class MyString {
private:
    char* data; // 指向动态分配的字符数组
    int len;    // 字符串长度

public:
    // 构造函数：使用new[] 分配内存并拷贝字符串
    MyString(const char* str);

    // 析构函数：使用delete[] 释放内存
    ~MyString();

    // 禁用拷贝构造函数
    MyString(const MyString&) = delete;

    // 禁用拷贝赋值运算符
    MyString& operator=(const MyString&) = delete;

    // 打印字符串
    void print() const;

    // 返回字符串长度
    int length() const;

    // 返回指定位置的字符
    char get(int index) const;
};

#endif

```

25 程设 II-周 5-课后 3

类型：编程题

题面

链表栈 (IntStack)

题目描述

实现一个 IntStack 类，使用链表存储整数。

类的要求：

1. 内部定义一个 Node 结构体，包含：
 - int val：节点存储的值
 - Node* next：指向下一个节点的指针
2. 包含一个私有成员 Node* head：指向栈顶节点
3. 提供构造函数 IntStack()：
 - 将 head 初始化为 nullptr
 - 输出信息："IntStack created"
4. 提供析构函数 ~IntStack()：
 - 释放所有节点占用的内存
 - 输出信息："IntStack destroyed"
5. 提供成员函数 void push(int val)：
 - 使用 new 创建一个新节点
 - 将新节点插入栈顶（头插法）
 - 输出信息："Push: X"（X 为压入的值）
6. 提供成员函数 void pop()：
 - 如果栈为空，输出 "Pop failed: stack is empty!"
 - 否则，删除栈顶节点，输出 "Pop: X"（X 为弹出的值）
7. 提供成员函数 int top() const：
 - 如果栈为空，输出 "Top failed: stack is empty!" 并返回 -1
 - 否则，返回栈顶元素的值
8. 提供成员函数 bool empty() const：
 - 如果栈为空返回 true，否则返回 false
9. 提供成员函数 void clear()：
 - 清空栈中所有元素（释放所有节点）

- 输出信息："Stack cleared"

输入数据

无输入

输出数据

```
=== IntStack 测试 ===  
IntStack created  
Push: 10  
Push: 20  
Push: 30  
Top: 30  
Pop: 30  
Top: 20  
Pop: 20  
Pop: 10  
Stack cleared  
Is empty? 1  
Pop failed: stack is empty!  
Top failed: stack is empty!  
IntStack destroyed
```

提交文件：IntStack.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>  
#include "IntStack.h"  
using namespace std;  
  
int main() {  
    cout << "=== IntStack 测试 ===" << endl;  
  
    IntStack s;  
  
    // 测试push 和top  
    s.push(10);  
    s.push(20);  
    s.push(30);  
    cout << "Top: " << s.top() << endl;  
  
    // 测试pop  
    s.pop();  
    cout << "Top: " << s.top() << endl;  
  
    // 继续pop  
    s.pop();  
    s.pop();  
}
```



```

// 测试clear
s.clear();

// 测试empty
cout << "Is empty? " << s.empty() << endl;

// 测试空栈操作
s.pop();
s.top();

return 0;
}

```

文件 IntStack.h :

```

#ifndef INTSTACK_H
#define INTSTACK_H

#include <iostream>
using namespace std;

class IntStack {
private:
    // 链表节点结构体
    struct Node {
        int val;
        Node* next;
    };

    Node* head; // 栈顶指针

public:
    // 构造函数
    IntStack();

    // 析构函数：释放所有节点
    ~IntStack();

    // 禁用拷贝构造函数
    IntStack(const IntStack&) = delete;

    // 禁用拷贝赋值运算符
    IntStack& operator=(const IntStack&) = delete;

    // 压栈
    void push(int val);

    // 出栈
    void pop();
}

```

```
// 获取栈顶元素 (不删除)
int top() const;

// 判断栈是否为空
bool empty() const;

// 清空栈
void clear();
};

#endif
```

25 程设 II-周 5-课后 4

类型：编程题

题面

二维点类 Point2D

实现一个 Point2D 类，表示二维平面上的点。

类的要求：

1. 包含两个私有成员：
 - const double x：x 坐标（常量，一旦初始化不可修改）
 - const double y：y 坐标（常量，一旦初始化不可修改）
2. 提供构造函数 Point2D(double x, double y)：
 - 使用**成员初始化器列表**初始化 x 和 y
 - 输出信息："Point2D created: (x, y)"（保留 2 位小数）
3. 提供成员函数 void print() const：
 - 输出点的坐标，格式："(x, y)"（保留 2 位小数）
4. 提供成员函数 double distance() const：
 - 计算并返回点到原点的距离，公式： $\sqrt{x*x + y*y}$
 - 输出信息："Distance to origin: d"（保留 2 位小数）

输入数据

一行，两个浮点数 x y，表示点的坐标

输出数据

三行：

1. 构造函数输出的创建信息
2. 点的坐标
3. 点到原点的距离

输入示例

3 4

输出示例

Point2D created: (3.00, 4.00)
(3.00, 4.00)
Distance to origin: 5.00

提交文件：Point2D.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include <iomanip>
#include "Point2D.h"
using namespace std;

int main() {
    cout << fixed << setprecision(2);

    double x, y;
    cin >> x >> y;

    Point2D p(x, y);
    p.print();
    p.distance();

    return 0;
}
```

文件 Point2D.h：

```
#ifndef POINT2D_H
#define POINT2D_H

#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;

class Point2D {
```

```
private:
    const double x; // 常量成员，必须使用初始化器列表
    const double y;

public:
    // 构造函数：使用成员初始化器列表
    Point2D(double x, double y);

    // 打印点的坐标
    void print() const;

    // 计算点到原点的距离
    double distance() const;
};

#endif
```

25 程设 II-周 5-课堂 1

类型：编程题

题面

动态整数存储

实现一个 DynamicInt 类，该类封装了一个动态分配的整数，用于练习 new 和 delete 的基本用法。

类的要求：

1. 包含一个私有成员 int* value，指向动态分配的整数
2. 提供构造函数 DynamicInt(int val)：
 - 使用 new 为 value 分配内存
 - 将传入的 val 存储到动态分配的空间中
 - 输出信息："DynamicInt created with value: X" (X 为传入的值)
3. 提供析构函数 ~DynamicInt()：
 - 使用 delete 释放动态分配的内存
 - 输出信息："DynamicInt destroyed"
4. 提供成员函数 int get() const：
 - 返回存储的整数值
5. 提供成员函数 void set(int val)：
 - 修改存储的整数值

输入数据

无输入

输出数据

DynamicInt created with value: 42

Value: 42

New value: 100

DynamicInt destroyed

提交文件：DynamicInt.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include "DynamicInt.h"
using namespace std;

int main() {
    DynamicInt num(42);
    cout << "Value: " << num.get() << endl;
    num.set(100);
    cout << "New value: " << num.get() << endl;
    return 0;
}
```

文件 DynamicInt.h：

```
#ifndef DYNAMICINT_H
#define DYNAMICINT_H

class DynamicInt {
private:
    int* value; // 指向动态分配的整数

public:
    DynamicInt(int val);
    ~DynamicInt();
    int get() const;
    void set(int val);
};

#endif
```

25 程设 II-周 5-课堂 2

类型：编程题

题面

实现一个 IntStack 类，使用**动态数组**保存栈中元素，练习类封装、构造/析构、new[]/delete[]、成员初始化器列表和 =delete 的用法。

类的要求

1. 包含以下私有成员：
 - int* data：指向动态分配的整型数组
 - int capacity：栈的最大容量
 - int topIndex：栈顶下标，空栈时为 -1
2. 提供构造函数：IntStack(int cap); 要求：
 - 使用**成员初始化器列表**初始化 capacity 和 topIndex
 - 使用 new[] 为 data 分配 capacity 个 int 的空间
 - 输出：Stack created with capacity: X
3. 提供析构函数：~IntStack(); 要求：
 - 使用 delete[] 释放空间
 - 输出：Stack destroyed
4. 禁止拷贝：

```
IntStack(const IntStack&) = delete;  
IntStack& operator=(const IntStack&) = delete;
```

5. 提供以下公有成员函数：

```
bool push(int x);    // 入栈成功返回 true，满栈返回 false  
bool pop();          // 出栈成功返回 true，空栈返回 false  
int top() const;     // 返回栈顶元素，若空栈返回 -1  
bool empty() const;  // 判空  
int size() const;    // 返回当前元素个数
```

6. push 和 pop 的输出要求：
 - 若 push(10) 成功，输出：Push: 10
 - 若栈满，输出：Push failed
 - 若 pop() 成功，输出：Pop successful
 - 若栈空，输出：Pop failed

main 函数测试要求

请在 main 函数中完成如下测试流程：

1. 创建容量为 3 的栈对象
2. 依次压入 10、20、30
3. 再压入一次 40，观察失败情况
4. 输出当前栈顶元素
5. 输出当前栈大小

6. 执行一次出栈
7. 再输出当前栈顶元素

示例输出

Stack created with capacity: 3
Push: 10
Push: 20
Push: 30
Push failed
Top: 30
Size: 3
Pop successful
Top: 20
Stack destroyed

提交文件：IntStack.cpp

平台提供的初始文件

文件 IntStack.hpp：

```
class IntStack {
private:
    int* data;    // 动态数组
    int capacity; // 栈容量
    int topIndex; // 栈顶下标，空栈时为-1

public:
    // 构造函数
    IntStack(int cap);

    // 析构函数
    ~IntStack();

    // 禁止拷贝
    IntStack(const IntStack&) = delete;
    IntStack& operator=(const IntStack&) = delete;

    // 基本操作
    bool push(int x);
    bool pop();
    int top() const;
    bool empty() const;
    int size() const;
};
```

文件 main.cpp：

```
#include <iostream>
#include "IntStack.hpp"
```

```
using namespace std;

int main() {
    IntStack s(3);

    s.push(10);
    s.push(20);
    s.push(30);
    s.push(40);

    cout << "Top: " << s.top() << endl;
    cout << "Size: " << s.size() << endl;

    s.pop();

    cout << "Top: " << s.top() << endl;

    return 0;
}
```

25 程设II-周 5-理论题

类型：选择题（20 题）

第 1 题

关于 C/C++ 程序内存布局，下列说法正确的是

- A. 自动变量通常存放在堆区
- B. 动态变量通常存放在栈区
- C. 自动变量通常存放在栈区
- D. 字符串字面量通常可以随意修改

答案：c

解析：函数参数、自动变量通常在栈区；动态变量（对象）在堆区；字符串字面量所在资源区不可改

第 2 题

下列代码中，最可能导致运行错误的是？

```
char *s1 = "Literal";
char s2[] = "Literal";
```

- A. 读取 s1[0]
- B. 输出 s2

- C. 修改 `s2[0] = 'X'`
- D. 修改 `s1[0] = 'X'`

答案：D

解析：这里 `s1` 是一个指针，它指向一个字符串字面量（String Literal）。在 C/C++ 中，字符串字面量通常存储在程序的只读数据段（.rodata）或常量区。这意味着这块内存是不可修改的。

第 3 题

关于 `new` 运算符，下列说法错误的是

- A. `new` 返回对应类型的指针
- B. `new` 分配失败时返回 `NULL`
- C. `new` 可以动态创建单个对象
- D. `new` 可以动态创建数组

答案：B

解析：`new` 返回该类型的指针，不是 `void*`；若内存分配失败，抛出异常，不是 `NULL`

第 4 题

执行下面代码后，正确的释放方式是

```
int *p = new int[10];
```

- A. `delete p;`
- B. `free(p);`
- C. `delete []p;`
- D. 不需要释放

答案：C

解析：动态分配的是数组，必须用 `delete[]` 释放。

第 5 题

关于 `delete`，下列说法正确的是

- A. `delete` 后指针会自动变成 `nullptr`
- B. `delete` 后指针仍保留原地址值，但已经失效
- C. `delete` 不能释放对象
- D. `delete[]` 不会调用析构函数

答案：B

解析：delete 释放后，指针的值仍是原来地址，但指针已无效；重复释放会出错。对象指针被 delete 时还会调用析构函数。

第 6 题

下列代码的输出结果是

```
int *p = new int[5]{1,3,5};  
for(int i = 0; i < 5; i++)  
    cout << p[i] << " ";  
delete []p;
```

- A. 1 3 5 1 3
- B. 1 3 5 0 0
- C. 1 3 5
- D. 运行时报错

答案：B

解析：数组用常量初始化列表时，若初始化项不足，后面的元素补零。

第 7 题

关于下面代码：

```
int *p = new int[3]{1,2,3};  
delete p;
```

- A. 这是正确写法
- B. 应该改为 delete []p;
- C. 编译一定报错
- D. delete p; 与 delete []p; 等价

答案：B

解析：如果动态分配了数组却用 delete p 释放，编译时可能没问题，运行时也可能不立刻报错，但实际上释放不完整，是错误写法

第 8 题

对象指针访问成员时，正确写法是

- A. p.Print();
- B. p->Print();
- C. *p.Print();
- D. &p->Print();

答案：B

解析：对象用 . 访问成员，指针用 -> 访问成员

第 9 题

对于下面代码：

```
Text str("hello");// 创建对象
Text* p = &str;
```

下列说法正确的是

- A. str 在堆上
- B. p 指向的对象需要手动 delete
- C. str 在栈上，离开作用域会自动析构
- D. p 是悬空指针

答案：C

解析：栈上对象会自动析构，取其地址赋给指针后，用 -> 访问即可，但不能对这个地址做 delete

第 10 题

下列哪一句最能体现“堆上对象不会自动析构”？

- A. Text str("abc");
- B. Text &r = str;
- C. Text *p = new Text("abc");
- D. Text arr[3];

答案：C

解析：new Text("on heap 1") 创建的对象在堆上，不会自动析构，必须显式 delete。

第 11 题

关于野指针问题，下列说法正确的是

- A. 两个指针指向同一块动态内存时，删除其中一个不会影响另一个
- B. 删除一个指针后，另一个指向同一地址的指针也会失效
- C. 删除一个指针后仍可正常解引用
- D. 重复 delete 同一块空间是安全的

答案：B

解析：两个指针指向同一块动态空间时，只要其中一个 delete 了，这块空间就失效了，另一个也一起失效。再次访问或再次释放都错误。

第 12 题

下面哪种情况会造成内存泄漏？

- A. `int *p = new int(7); delete p;`
- B. `int *p = new int(7); p = nullptr;`
- C. `int x = 7; int *p = &x;`
- D. `int *p = new int(7); cout << *p; delete p;`

答案：B

解析：`int* p = new int(7); p = nullptr;` 会造成内存泄漏，因为原先那块动态空间失去了唯一指针，无法再释放。

第 13 题

下列关于内存泄漏的说法错误的是

- A. 指针离开作用域前若未释放动态空间，可能造成泄漏
- B. 异常导致 `delete` 未执行，也可能造成泄漏
- C. 只要程序结束，内存泄漏就一定不是问题
- D. 丢失指向动态空间的指针会造成泄漏

答案：C

解析：内存泄漏会导致资源耗尽。虽然某些环境下程序结束后操作系统会回收内存，但从程序设计和长期运行角度看，泄漏依然是错误。

第 14 题

关于 `=default`，下列说法正确的是

- A. 可以用于任意普通成员函数
- B. 可以用于让编译器生成特殊成员函数
- C. 使用 `=default` 后还要自己写函数体
- D. `B(int=0)=default;` 一定合法

答案：B

解析：`=default` 仅用于编译器可自动生成的特殊成员函数，如默认构造、析构、拷贝构造、赋值运算等；不能用于普通函数，也不能用于带默认实参的这种构造形式。

第 15 题

当一个类已经声明了有参构造函数时，编译器

- A. 一定还会自动生成默认构造函数
- B. 不会自动生成默认构造函数
- C. 一定不生成析构函数
- D. 一定不生成拷贝构造函数

答案：B

解析：当我们声明有参构造函数时，编译器不会创建默认构造函数。

第 16 题

=delete 的主要作用是

- A. 释放堆空间
- B. 禁止某个函数被使用
- C. 让函数更快
- D. 替代析构函数

答案：B

解析：如果函数用 = delete 定义为弃置函数，则任何对它的使用都是非良构的，也就是程序无法编译。其作用就是显式禁用该函数。

第 17 题

下面代码中被禁用的是

```
class A {  
public:  
    A(int x): m(x) {}  
    A(const A&) = delete;  
    A& operator=(const A&) = delete;  
    int m;  
};
```

- A. 默认构造
- B. 析构函数
- C. 拷贝构造和拷贝赋值
- D. 成员变量初始化

答案：C

解析：代码中明确把复制构造函数和复制赋值运算符写成了 =delete，所以它们被禁止使用。

第 18 题

关于删除函数，下列说法正确的是

- A. 删除函数可以先声明，之后再定义为 =delete
- B. 被删除函数运行时会报错
- C. 删除函数定义必须是函数的首次声明
- D. =delete 只能用于构造函数

答案：C

解析：删除函数的定义必须是函数的首次声明。

第 19 题

关于成员初始化器列表，下列说法正确的是

- A. 它写在函数体内部
- B. 它写在构造函数参数列表和函数体之间
- C. 它只能初始化一个成员
- D. 它只能用于内置类型

答案：B

解析：成员初始化器列表位于“构造函数名字、参数列表和函数体之间”，以冒号开头

第 20 题

下列哪种成员必须使用初始化器列表进行初始化？

- A. 普通局部变量
- B. const 成员或引用成员
- C. 所有 int 成员
- D. 静态成员函数

答案：B

解析：以下情况必须使用初始化器列表：常量成员、引用类型成员、没有无参构造函数的类成员。这里最典型的是 const 成员和引用成员。
